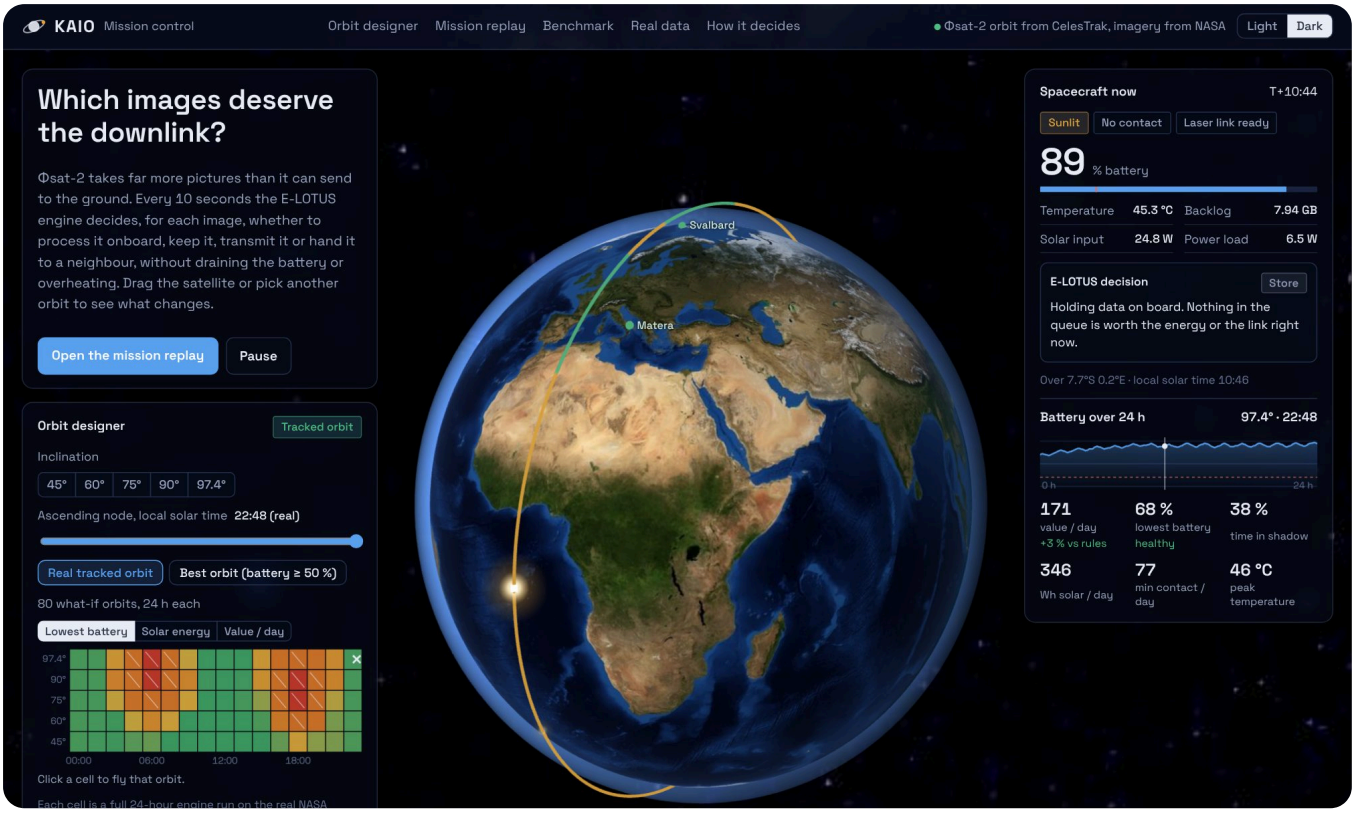
KAIO

IASTAM 6.0 Technical Challenge · Track 1 · Problem 1 “Process or Transmit?”

KAIO: deciding onboard which images deserve the downlink

Final technical report. A real-orbit, real-data evaluation of the E-LOTUS onboard decision engine, its safety guard and its embedded implementation.



Team KAIO: Nadhem Selmi · Mahmoud Amri

15 October 2026 · Code: github.com//kaio · All numbers in this report are generated from `results/summary.json` by `docs/build_report.py`.

Summary

An Earth-observation satellite produces far more data than it can downlink. For each image it must decide to **process** it onboard, **keep** it, **transmit** it, or **offload** it to a neighbour over a laser link, within its energy, thermal, storage and contact budgets. KAIO is an onboard decision system that makes this choice every 10 seconds with the **E-LOTUS** engine: a value estimate built only from onboard information, a Lyapunov drift-plus-penalty score, an independent safety guard and a one-line explanation per decision.

We evaluate it on the **real orbit of ESA's Φsat-2** (CelesTrak TLE, SGP4) and on **20 days of real NASA imagery** sampled under its ground track (VIIRS, MODIS, FIRMS), against five reference strategies, in eight scenarios, with 95 % confidence intervals.

85.3 %	**+3.7 pts**	**0**	**0.16 ms**
of the reachable value delivered (± 0.6, 20 real days)	over a tuned rule engine with the same information; better on 20/20 days	load-sheds or hardware throttles; time above 45 °C 0.1 %	median decision time (laptop), for a 10 s decision cycle

The main finding from real data is that **raw image entropy misranks scenes.** Clouds are the most textured scenes and 64 % of all scenes, so ranking by entropy alone captures 27 % of the value in the top 30 %, worse than random (30 %). Multiplying entropy by an onboard land-mask prior reaches 79 %, close to a label-aware oracle (82 %). E-LOTUS wins 6 of 8 stress scenarios. It loses two, a slower GPU and a degraded radiator, where the rule engine scores higher only by running above the 45 °C limit for 37–81 % of the day.

Contents

1. Problem and performance indicators
2. Real data: orbit and Earth observations
3. System architecture
4. The E-LOTUS decision engine
5. Implementation
6. Evaluation protocol
7. Results against the performance indicators
8. Robustness and ablations
9. Feasibility and embedded implementation
10. Methodology, planning and roadmap
11. Limitations
12. References and AI-use statement

1. Problem and performance indicators

Problem 1 asks: how can the system decide automatically, for each piece of data or task, whether to process it onboard, store it for later, or transmit it to the ground, so as to maximise scientific or operational value under the available resources?

Constraints: available energy, CPU/GPU load, memory, storage, data priority, communication windows, bandwidth and transmission delay.

The challenge evaluates five indicators. We map each one to measured quantities.

Indicator	How KAIO measures it
Quality of decisions	Value delivered to the ground as a share of the *reachable* value (each image delivered at its first ground pass with unlimited resources), and fire alerts delivered while still useful.
Rate of completed tasks	Share of images that are delivered (raw or as a product) or triaged onboard.
Estimated energy consumption	Payload energy per day (GPU, radio, laser), lowest battery level, payload load-shed time.
Latency	Time from image acquisition to delivery on the ground (mean and 95th percentile).
Resource utilisation	GPU duty cycle, downlink use during passes, storage backlog, and time above the thermal limit.

The ”reachable” ceiling is physically unattainable (zero processing and transfer time), so 100 % is an upper bound, not a target.

2. Real data: orbit and Earth observations

2.1 Orbit and geometry

The spacecraft flies the real orbit of **Φsat-2** (NORAD 60470). Its CelesTrak two-line element set is propagated with **SGP4** every 10 s. This gives about 472 km altitude, a 94-minute period and about 35-minute eclipses. An independent fourth-order Runge–Kutta integrator with the J2 term agrees within 25 km over 6 h (automated test).

From the position we derive:

- sun or eclipse, with a cylindrical shadow model and a low-precision solar ephemeris;
- ground contacts with KSAT **Svalbard** and ASI **Matera** above 10° elevation, about 79 min per day;
- line of sight to two in-plane neighbours for laser offload.

2.2 Earth observations under the real ground track

For 20 days (26 Aug – 14 Sep 2026), every 30 s along the real ground track, a 32 × 32-pixel block (about 313 km) was cut from the NASA GIBS daily global mosaics and reduced to a few numbers:

Quantity	NASA GIBS layer	Use
Day quick-look entropy	VIIRS NOAA-20 Corrected Reflectance, true colour	onboard value estimate
Night quick-look entropy	VIIRS NOAA-20 Brightness Temperature, band I5 night	onboard value estimate
Fire pixels	VIIRS NOAA-20 Thermal Anomalies 375 m (FIRMS)	alert flag (priority 4)
Cloud fraction	MODIS Aqua Cloud Fraction, day / night (decoded from the colour map)	hidden ground truth only
Land fraction	OpenStreetMap land/water map	onboard prior + ground truth

The result is an atlas of 57,600 real footprints (`data/elotus_gibs_atlas.bin` , sampler `data/gibs_sampler.js`). Each simulated acquisition receives the real scene that was under Φsat-2 at that time. The hidden ground-truth value is high for cloud-free land, low for cloud, ocean or polar night, and very high for fires. It is used only for scoring. A test shuffles it and checks that the engine's decisions do not change.

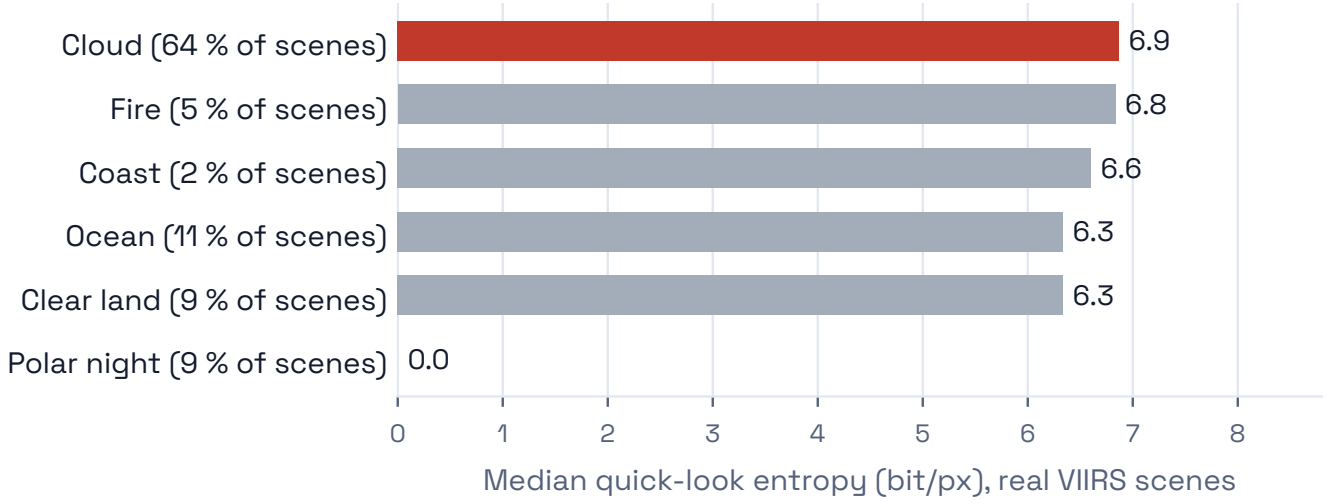


Fig. 1. Median quick-look entropy by real scene class. Clouds are the most textured scenes and the most common.

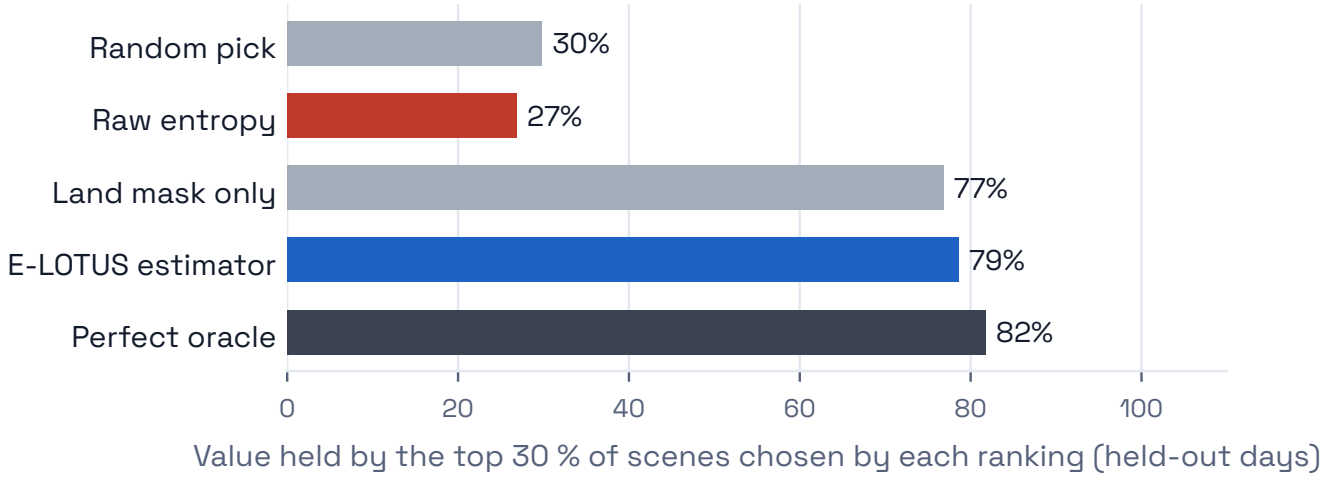


Fig. 2. Value captured by the top 30 % of scenes on held-out days. Raw entropy is worse than random; the E-LOTUS estimator is within 3 points of an oracle.

3. System architecture

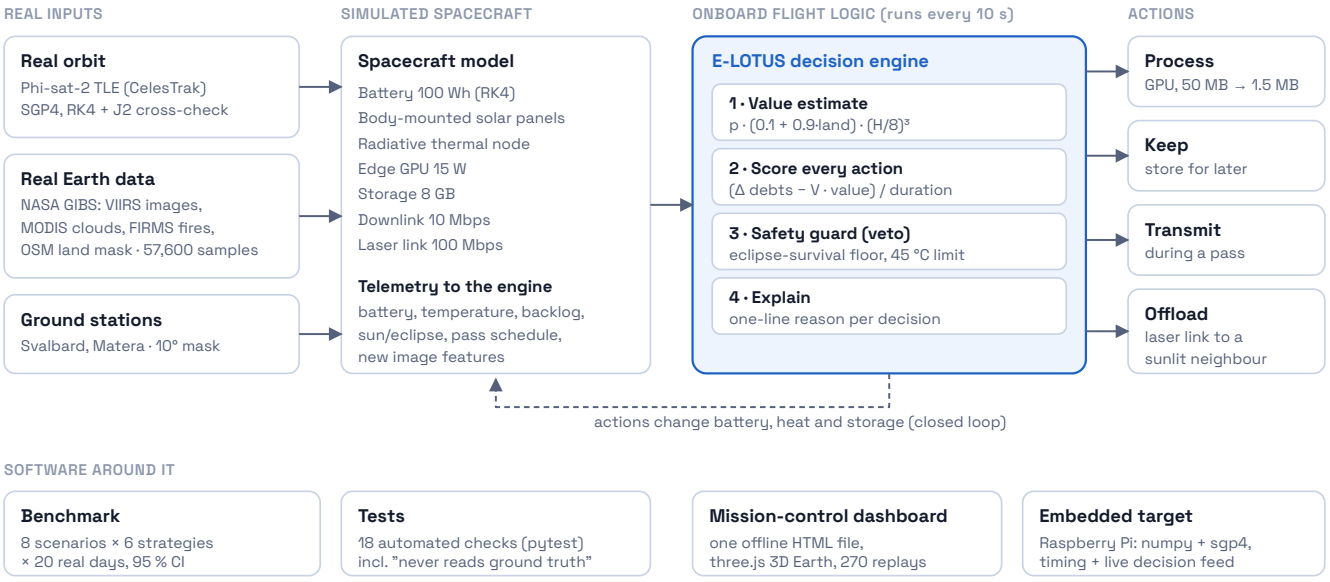


Fig. 3. KAIO architecture. Real inputs drive a simulated spacecraft. The E-LOTUS flight logic reads its telemetry every 10 s and commands one of four actions. The surrounding software benchmarks, tests, visualises and runs the same code on embedded hardware.

The design separates three concerns, as in flight software:

- **the world:** orbit, data and spacecraft physics, all policy-independent;
- **the optimiser:** the E-LOTUS score;
- **the protection:** an independent safety guard.

Every controller, including the baselines, sees exactly the same orbit, images and hardware limits. Only the decision rule changes.

Spacecraft model (simulated)

Subsystem	Model and main parameters
Power	100 Wh battery integrated with RK4, 0.95 charge/discharge efficiency, CV taper above 90 %; body-mounted panels (20 W per face at normal incidence); 6.5 W platform bus; payload load-shed below 20 Wh.
Thermal	Lumped node, radiator to deep space (ε = 0.85, A = 0.045 m², Stefan–Boltzmann), solar and Earth-IR loads; operating limit 45 °C, hardware throttle 60 °C.
Compute	Edge-AI module, 15 W active (Jetson-Orin class), 1 TFLOP/s effective; processing reduces an image to 3 % of its size and keeps 90 % of its information.
Links and storage	10 Mbps downlink, 100 Mbps laser link to a sunlit neighbour with limited spare compute, 8 GB storage.
Workload	Poisson acquisitions (one per 45 s in sunlight, one per 120 s in eclipse), about 50 MB per image, 1.5 h deadline for fire alerts, 12 h otherwise.

4. The E-LOTUS decision engine

4.1 Value estimate (onboard)

For each image, the engine uses two numbers available onboard:

- the Shannon entropy H of a quick-look thumbnail;
- the land fraction ℓ of the footprint, looked up in a static land/water mask indexed by the onboard position.

It also uses the priority p, which is 4 for a fire alert and 1 otherwise.

$$\hat{w} = p \cdot (0.1 + 0.9\,\ell) \cdot (H / 8)^3 \text{ for optical scenes}, \quad \hat{w} = p \cdot (0.1 + 0.9\,\ell) \text{ for thermal-IR night scenes}$$

4.2 Scoring: drift-plus-penalty

The engine keeps three normalised "debts" small:

- the storage backlog q/Q;
- the energy debt D_E below an eclipse-aware battery floor $E_{min}(t)$;
- the heat debt D_T above T_{safe} = 45 °C.

$$L = \tfrac{1}{2}\left[(q/Q)^2 + (D_E/5\ \text{Wh})^2 + (D_T/5\ \text{K})^2 \right], \quad \Phi_i(a) = \left[\Delta L_i(a) - V \cdot U_i(a) \right] / \tau_i(a)$$

For each stored image i and action a (process, transmit, offload), the engine predicts the change ΔL relative to keeping the image. It predicts it over the action duration τ, using a linear energy budget and a first-order thermal model. U is the delivered value: $\hat{w}$ × retained information × a freshness term that decays with age until the deadline, minus an energy price that is zero when the solar array would otherwise be shunted. Free resources are filled greedily with the most negative Φ. $\mathbf{V}$ (0.3) is the single trade-off knob.

This is the drift-plus-penalty rule of Lyapunov optimisation [1]. We use it as a principled scoring rule. Its textbook performance bound assumes stationary arrivals and exact per-slot minimisation, which an orbit does not provide, so all claims in this report are empirical.

4.3 Safety guard

An independent layer checks each proposed job before it starts, like FDIR in flight software. It blocks the job if either of these would happen at the end of the job:

- the battery would fall below the energy needed to carry the platform through the current or next eclipse plus a reserve;
- the computer would exceed 45 °C. Fire alerts may use the margin up to the 60 °C hardware limit.

Blocked jobs are logged with their reason.

4.4 Storage and explanations

When storage is full, the engine deletes the items with the lowest expected value per megabyte first. The rule-based baselines get the same rule, for fairness. Every started job carries a one-line reason built only from onboard quantities, for example: *"Process now: est. value 0.68 (optical H 7.5, land 79 %). 20 MB raw → 0.6 MB product, next pass in 476 min; battery 69 Wh after (floor 20), 9.3 °C."*

5. Implementation

Layer	Technology	Content
Flight logic and simulation	Python 3, numpy, sgp4	Package `kaio/` : `orbit` , `workload` , `estimator` , `physics` , `safety` , `engine` , `baselines` , `simulator` , `scenarios` . Every assumption is in `config.py` .
Verification	pytest	18 tests: SGP4 vs RK4/J2, physics bounds, entropy limits, guard vetoes, determinism, *engine never reads ground truth*, image bookkeeping, safety properties.
Benchmark	Python multiprocessing, scipy	`benchmark.py` : 8 scenarios × 6 strategies × 20 days plus ablations (1,360 runs), 95 % CIs, paired days-won.
Dashboard	HTML, CSS, JavaScript, three.js, Canvas	One self-contained offline file: 3D Earth with the real orbit, orbit designer (80 what-if orbits), 24 h replay of 270 what-if missions, benchmark, real-data findings; light/dark mode.
Embedded target	Raspberry Pi, Python, numpy, sgp4	`onboard/pi_benchmark.py` (timing, CPU, memory) and `onboard/live_demo.py` (live decision feed with reasons).
Exploration	Streamlit, Plotly	`app.py` : run new simulations interactively.

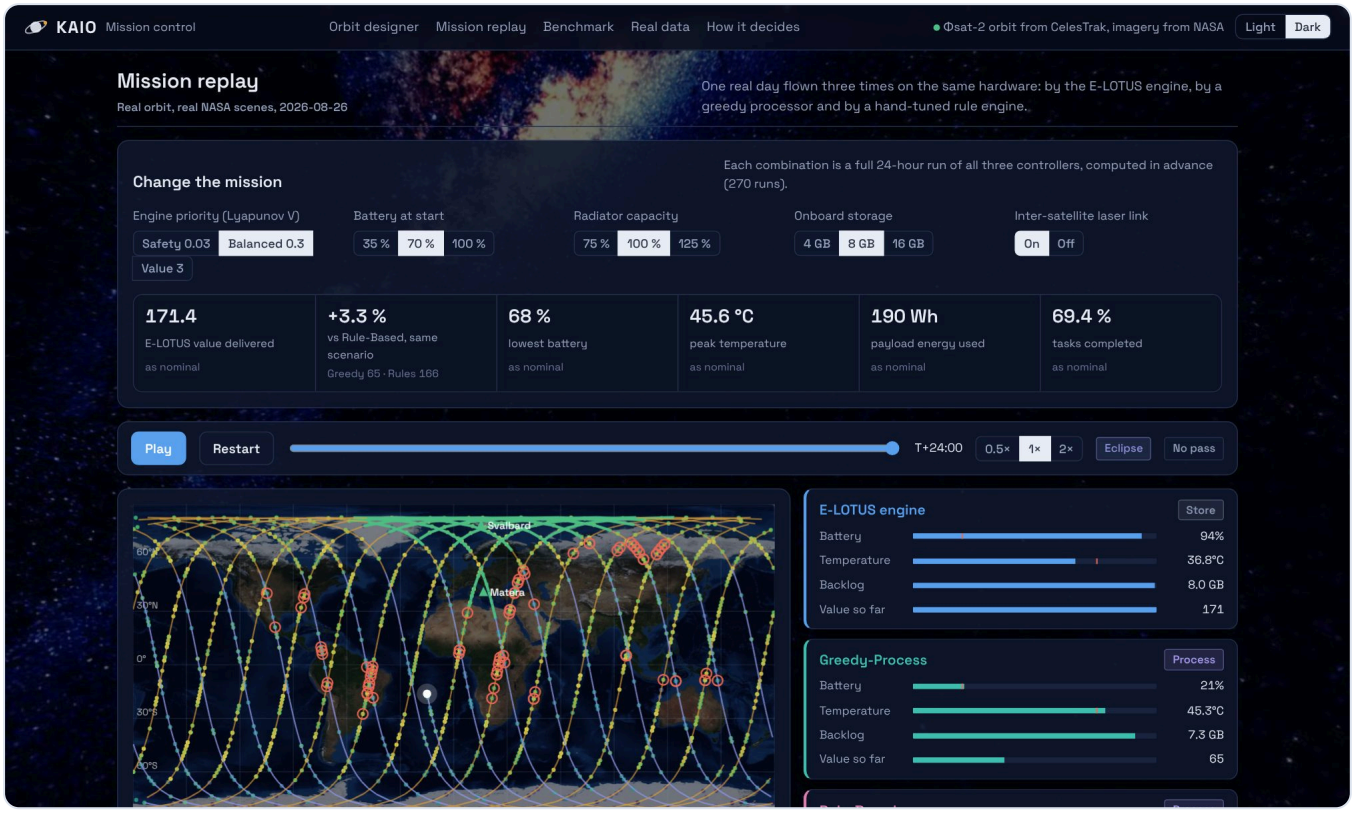


Fig. 4. Dashboard: one real day replayed by E-LOTUS, a greedy processor and a rule engine. The controls on top switch to any of 270 pre-computed what-if missions.

6. Evaluation protocol

- **Days:** 20 real days, each an independent 24 h run starting at 00:00 UTC on 2026-08-26 + *d*. The workload seed is 2026 + *d*, and every strategy sees the same images on a given day.
- **Strategies:**
  - **Always-Transmit** (bent pipe);

    - **Greedy-Process** (process everything);
    - **Store-and-Forward** (no onboard compute);
    - **Rule-Based**, a hand-tuned onboard rule engine using the *same* value estimate, with battery and temperature thresholds;
    - **Bandwidth-Rules**, Rule-Based plus pass-capacity awareness.
- **Scenarios:** nominal, plus seven single-resource degradations fixed in advance and applied without retuning any controller.
- **Statistics:** mean ± 95 % confidence interval (Student t over days) and the number of days on which E-LOTUS beats each baseline.
- **Tuning hygiene:** V and the debt weights were selected on a different workload seed. The thermal limit and the fire-alert exception are design rules with no tuned value.

7. Results against the performance indicators

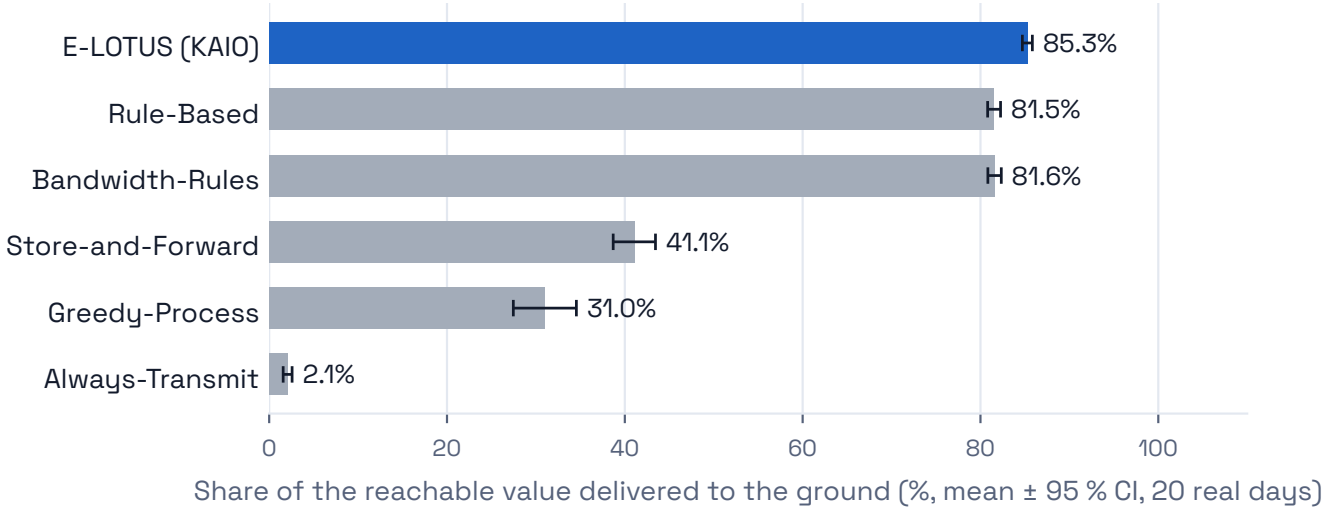


Fig. 5. Quality of decisions, nominal scenario. E-LOTUS 85.3 ± 0.6 %, Rule-Based 81.5 ± 0.7 %, Bandwidth-Rules 81.6 ± 0.8 %. The intervals do not overlap, and E-LOTUS is better on 20 of 20 days.

Indicator (Problem 1)	Metric	E-LOTUS	Rule-Based	Greedy	Store & fwd	Always-Tx
Quality of decisions	Value delivered, % of the reachable value	**85.3**	81.5	31.0	41.1	2.1
	Fire alerts delivered while still useful (%)	**72.7**	72.8	21.4	63.7	0.0
Rate of completed tasks	Images delivered or triaged onboard (%)	**71.9**	47.8	47.2	7.5	7.4
Estimated energy consumption	Payload energy per day (Wh)	**193**	159	264	52	52
	Lowest battery level (%)	**69**	68	19	69	69
Latency	Mean generation-to-ground latency (min)	**120**	134	239	73	160
	95th-percentile latency (min)	**442**	473	477	163	233
Resource utilisation	GPU duty cycle (%)	**44**	33	71	0	0
	Downlink use during passes (%)	**85**	100	28	100	100
Safety	Time above the 45 °C limit (%)	**0.1**	0.0	53.4	0.0	0.0
	Time with payload load-shed (%)	**0.0**	0.0	18.5	0.0	0.0

Nominal scenario, mean of 20 real days. Underlined: best value where higher or lower is clearly better.

Reading the indicators

- **Quality of decisions:** E-LOTUS delivers the most value (+3.7 pts over the rule engine) and matches the best strategies on fire alerts (72.7 %).
- **Completed tasks:** 71.9 % against 47.8 % for the rule engine. The laser offload in eclipse and value-aware storage management keep more images moving.

- **Energy:** E-LOTUS spends more payload energy (193 vs 159 Wh/day), so its value per Wh is lower. This extra energy comes from solar power that would otherwise be shunted: the battery never drops below 69 %, as with the rule engine, and there is no load-shed. Greedy-Process shows the opposite failure: it drains the battery to 19 % and sheds the payload 19 % of the time.
- **Latency:** E-LOTUS has the lowest mean latency of the processing strategies (120 min vs 134 min). Store-and-Forward is faster only because it delivers very few images.
- **Resource utilisation:** E-LOTUS uses the GPU more (44 %) and fills 85 % of pass capacity. The remaining capacity goes unused because the products it sends are small. Time above 45 °C is 0.1 % (fire alerts only), against 53 % for Greedy.

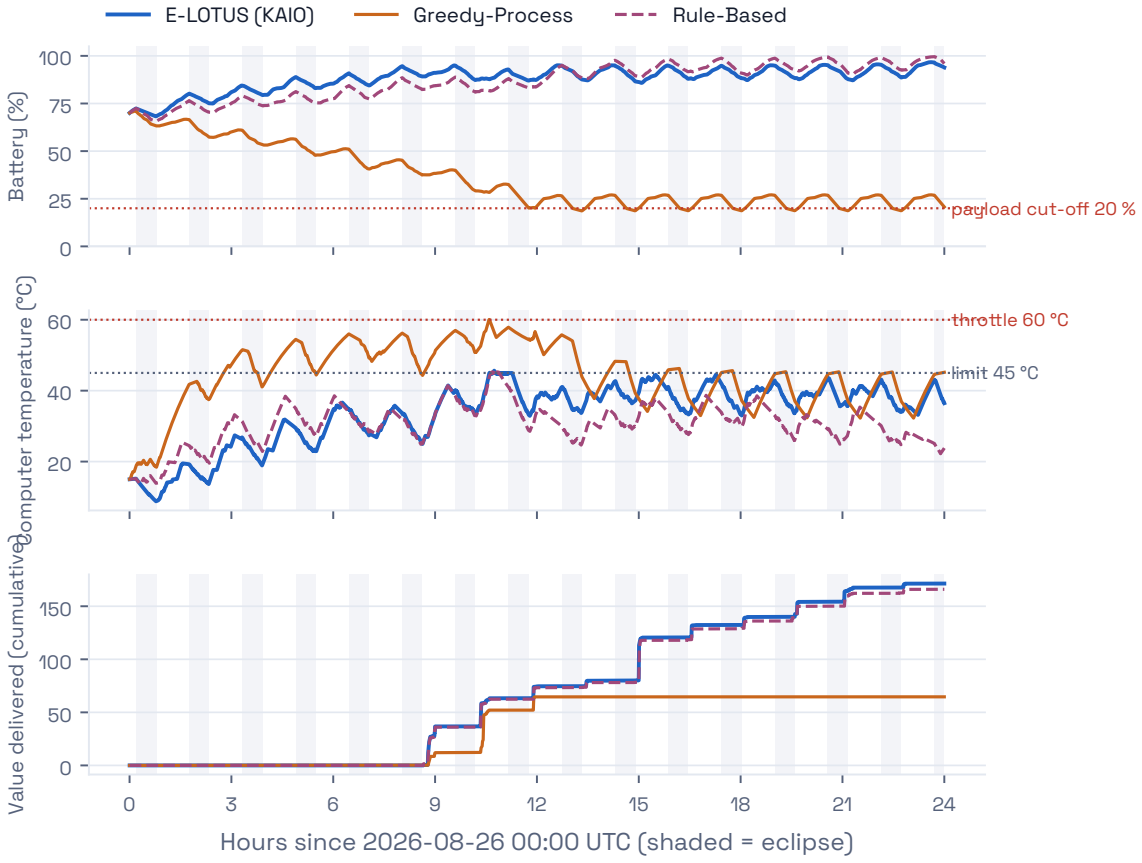


Fig. 6. One real day (2026-08-26). Greedy processing drains the battery to the payload cut-off and runs hot. E-LOTUS keeps the battery high and the computer under 45 °C, and delivers the most value.

8. Robustness and ablations



Fig. 7. Eight scenarios, 20 real days each, no retuning.

Scenario (one resource degraded)	E-LOTUS	Rule-Based	Bandwidth-Rules	E-LOTUS better on	Rule-Based time > 45 °C
Nominal	**85.3 ± 0.6**	81.5 ± 0.7	81.6 ± 0.8	20/20 days	0 %
Solar array -30 %	**83.3 ± 0.5**	81.1 ± 0.8	76.0 ± 1.8	20/20 days	0 %
Small battery (40 Wh) + solar -30 %	**80.1 ± 0.7**	2.9 ± 0.2	2.9 ± 0.2	20/20 days	0 %
Storage / 3 (2.7 GB)	**86.6 ± 0.5**	81.5 ± 0.7	81.6 ± 0.8	20/20 days	0 %
Downlink / 2, 20 deg mask	**84.5 ± 0.6**	80.5 ± 0.8	80.3 ± 1.0	20/20 days	0 %
Fire season (3x alerts)	**86.8 ± 0.5**	84.5 ± 0.7	83.9 ± 1.1	20/20 days	0 %
GPU 2.5x slower	**79.6 ± 0.8**	80.6 ± 0.8	75.3 ± 1.8	2/20 days	37 %
Radiator -25 %	**71.2 ± 1.4**	79.8 ± 0.9	76.9 ± 1.9	0/20 days	81 %

Wins (6/8). E-LOTUS beats both rule engines on every day of six scenarios. With a 40 Wh battery and weaker panels, the rule engine's fixed 50 % battery threshold coincides with the shutdown floor and it collapses (2.9 %). E-LOTUS's eclipse-survival floor is computed from the orbit and adapts automatically (80.1 %).

Losses (2/8). With a 2.5× slower GPU or a 25 % weaker radiator, the rule engine scores higher. It does so by running above 45 °C for 37 % and 81 % of the day. If operators allow 60 °C for every job, E-LOTUS reaches 82.1 % in the radiator case, above the rule engine. The thermal limit is an operator setting, not a limit of the method.

Ablations

Variant	Nominal	Solar −30 %	Battery 40 Wh	Radiator −25 %
Full E-LOTUS	**85.3**	**83.3**	**80.1**	**71.2**
Without entropy (land mask only)	87.1	78.3	66.3	69.3
Without land mask (raw entropy)	83.5	51.7	41.7	63.4
Without laser offload	83.7	81.6	77.8	67.0
Without eclipse-aware battery floor	85.3	82.9	79.2	71.2

Main points:

- **The land-mask prior is the key ingredient.** Raw entropy alone loses 2 to 38 points.
- **Entropy pays off when resources are scarce** (+2 to +14 points). In the nominal case the land mask alone is slightly better, because there is enough energy to process every land scene.
- **Laser offload is a modest extension:** 1.6 to 4.2 points.
- **The eclipse-aware floor matters under energy stress:** +0.4 to +0.9 points.

9. Feasibility and embedded implementation

Every input the engine uses exists on a real satellite:

- battery and temperature housekeeping;
- a pass schedule computed on the ground and uplinked;
- eclipse times from the onboard propagator;
- a static land/water mask indexed by GNSS position;
- a thumbnail entropy.

The engine needs no GPU and no training; its only dependencies are numpy and sgp4. ESA's Φsat-1 already discarded cloudy images onboard [4]; KAIO flies the real orbit of its successor.

Measured on development laptop (x86_64); Raspberry Pi run pending	Value
E-LOTUS decision time (median / p99 / worst)	0.16 / 0.55 / 3.0 ms
Decision cycle required by the satellite	every 10 s
CPU used by the engine over a full real day	0.002 % of one core
Quick-look entropy, one 32 × 32 thumbnail	15 µs
Peak memory, whole program including the simulator	48 MB

Regenerate with `python3 onboard/pi_benchmark.py` on the target, then `python docs/build_report.py`.

10. Methodology, planning and roadmap

How we worked

1. **Phase 2, two independent prototypes:** E-LOTUS (real orbit, NASA data, Lyapunov engine, dashboard) and a value-aware pricing engine (tested package, calibrated scenarios, explanations). Both found that a value-aware engine beats tuned rules by a few points.
2. **Review and merge into KAIO:** we kept the real-data world, the E-LOTUS engine and the dashboard, and adopted the tested package structure, the explanations, the drop rule, the bandwidth-aware

baseline and the statistical protocol.

3. **Fixes found by the review:** the previous engine spent 10 % of the time above 45 °C. The independent safety guard reduces this to 0.1 %. All claims were re-worded as empirical.
4. **Verification:** 18 automated tests, then the full benchmark (1,360 runs), regenerated end to end by scripts.
5. **Embedded check:** the same code runs on a Raspberry Pi with timing instrumentation.

Roadmap to flight

When	Milestone	Success criterion
Done	Engine, safety guard, real-data simulation, benchmark, dashboard, embedded run	All results in this report reproducible from scripts
+3 months	Hardware-in-the-loop on a flight-like onboard computer; imperfect pass and eclipse forecasts; sensor noise	Same ranking of strategies with realistic forecast errors
+6 months	Small onboard cloud-screening network replacing the static land prior; comparison with a hindsight-optimal schedule	Triage within 3 points of the oracle; optimality gap quantified
+12 months	Coordination across a constellation; partnership for a CubeSat flight experiment	Flight-software integration plan and test campaign

11. Limitations

- The spacecraft hardware and the ground-truth value model are assumptions, not a specific mission. The value of a scene (cloud-free land, fires) would be calibrated by operators.
- Pass and eclipse forecasts come from the same propagated orbit and are therefore exact.
- The laser-link neighbour is modelled. Without it, E-LOTUS still beats the rule engines.
- The TLE epoch is 26 Sep 2026, so August positions are propagated about one month backwards (kilometre-level error, small against 313 km footprints).
- The imagery days are shared between tuning and evaluation; only the acquisition streams differ.
- Baselines are heuristics. A hindsight-optimal schedule is future work.

12. References and AI-use statement

1. M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*, Morgan & Claypool, 2010.
2. D. A. Vallado, P. Crawford, R. Hujsak, T. S. Kelso, "Revisiting Spacetrack Report #3", AIAA/AAS Astrodynamics Specialist Conference, 2006.
3. NASA Global Imagery Browse Services (GIBS); NASA FIRMS active-fire data; CelesTrak element sets.
4. G. Giuffrida et al., "The Φ-Sat-1 mission: the first on-board deep neural network demonstrator for satellite Earth observation", *IEEE Transactions on Geoscience and Remote Sensing*, 2022.
5. C. E. Shannon, "A Mathematical Theory of Communication", *Bell System Technical Journal*, 1948.

AI-use statement. We used Claude (Anthropic) as a coding and writing assistant: to review and merge the two prototypes, implement the safety guard, tests and benchmark scripts, redesign the dashboard, write the embedded scripts, and draft this report and the presentation. The team chose the design, checked every result against the code, and is responsible for the content.